Predicting Vehicle Prices using Regression Models –

1. Data understanding

1.1 Data Loading

Import necessary libraries, load the dataset.

verify the column names, datatypes and data in each column

```
# Importing necessary libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_squared_error, r2_score
```

1.1.1

Load the dataset

```
# Load the data
car_df = pd.read_csv('/content/Car_Price_data.csv')
car_df.head()
```

```
print(car_df.shape)

car_df.info()
```

```
(15915, 23)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15915 entries, 0 to 15914
Data columns (total 23 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   make_model           15915 non-null  object
 1   body_type            15915 non-null  object
 2   price                15915 non-null  int64
 3   vat                  15915 non-null  object
 4   km                   15915 non-null  float64
 5   Type                 15915 non-null  object
 6   Fuel                 15915 non-null  object
 7   Gears                15915 non-null  float64
 8   Comfort_Convenience  15915 non-null  object
 9   Entertainment_Media  15915 non-null  object
 10  Extras               15915 non-null  object
```

```
 11  Safety_Security      15915 non-null  object
 12  age                  15915 non-null  float64
 13  Previous_Owners      15915 non-null  float64
 14  hp_kW                15915 non-null  float64
 15  Inspection_new       15915 non-null  int64
 16  Paint_Type           15915 non-null  object
 17  Upholstery_type      15915 non-null  object
 18  Gearing_Type         15915 non-null  object
 19  Displacement_cc      15915 non-null  float64
 20  Weight_kg            15915 non-null  float64
 21  Drive_chain          15915 non-null  object
 22  cons_comb            15915 non-null  float64
dtypes: float64(8), int64(2), object(13)
memory usage: 2.8+ MB
```

2. Analysis and Feature Engineering

2.1 Preliminary analysis and Frequency Distribution

2.1.1 Check and fix the missing values

There are no missing values in any columns

```
# Proportion of missing values (percentage)
missing_pct = car_df.isnull().mean() * 100

# Sort in descending order
missing_pct = missing_pct.sort_values(ascending=False)

print(missing_pct)
```

```
make_model            0.0
body_type             0.0
price                 0.0
vat                   0.0
km                    0.0
Type                  0.0
Fuel                  0.0
Gears                 0.0
Comfort_Convenience   0.0
Entertainment_Media   0.0
Extras                0.0
Safety_Security       0.0
age                   0.0
Previous_Owners       0.0
hp_kW                 0.0
Inspection_new        0.0
Paint_Type            0.0
Upholstery_type       0.0
Gearing_Type          0.0
Displacement_cc       0.0
Weight_kg             0.0
Drive_chain           0.0
cons_comb             0.0
dtype: float64
```

From the features, identify the target feature and numerical and categorical predictors.

Target Feature –
Price

Numerical Predictors –
Km, age, Previous_Owners, hp_kW, Gears, Displacement_cc, Weight_kg, cons_comb

Categorical Predictors -
Make_model, body_type, vat, Type, Fuel, Comfort_Convenience, Entertainment_Media, Extras, Safety_Security, Inspection_new, Paint_Type, Upholstery_type, Gearing_Type, Drive_chain.

2.1.2 – Identify numerical Predictors and plot histogram

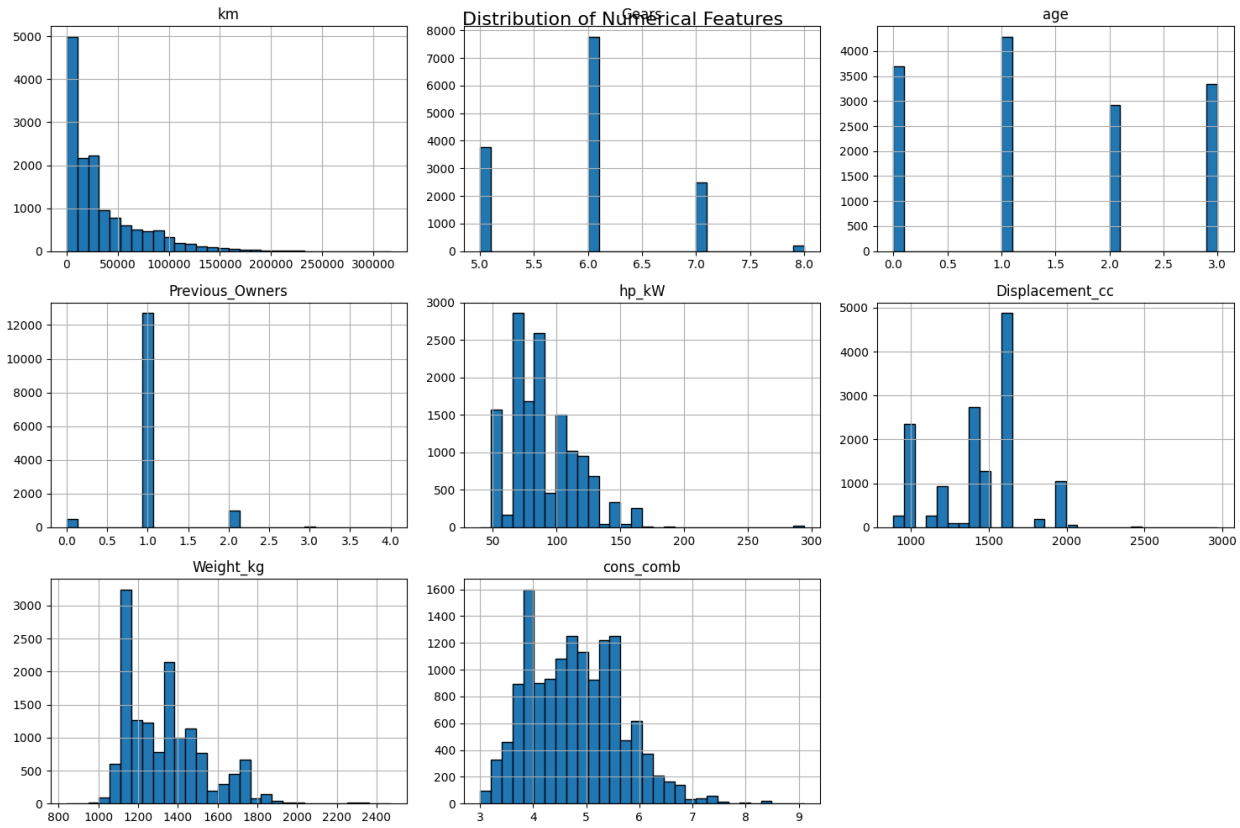
```
numerical_features = car_df.select_dtypes(include=['int64', 'float64']).columns
#print(numerical_features)

numerical_features = numerical_features.drop('price')
print(numerical_features)


car_df[numerical_features].hist(
    figsize=(15, 10),
    bins=30,
    edgecolor='black'
)

plt.xticks(rotation=0)
plt.tight_layout()
plt.show()

plt.suptitle("Distribution of Numerical Features", fontsize=16)
plt.show()
```



2.1.3 – Identify the categorical predictors and plot their frequency distributions

```
# Select categorical columns
categorical_cols = car_df.select_dtypes(include=['object']).columns
print(categorical_cols)

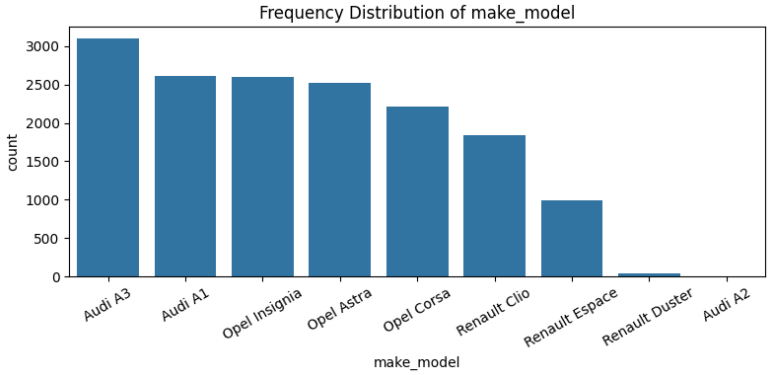
#Checking Frequency Distribution
for col in categorical_cols:
    print(f"\nColumn: {col}")
    print(car_df[col].value_counts(normalize=True) * 100)
```

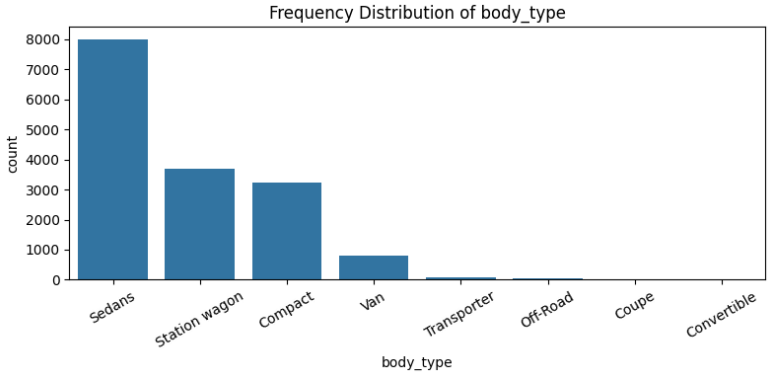
```
#Visualise the distribution
cols = ['make_model', 'body_type', 'vat', 'Type', 'Fuel',
        'Paint_Type','Upholstery_type', 'Gearing_Type', 'Drive_chain']

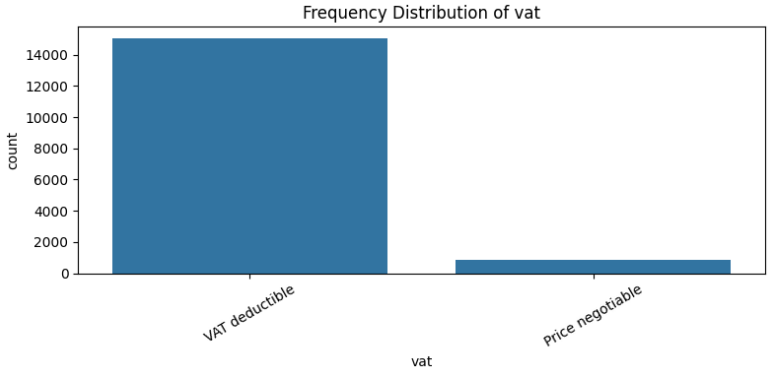

     # 'Safety_Security', 'Comfort_Convenience','Entertainment_Media', 'Extras',

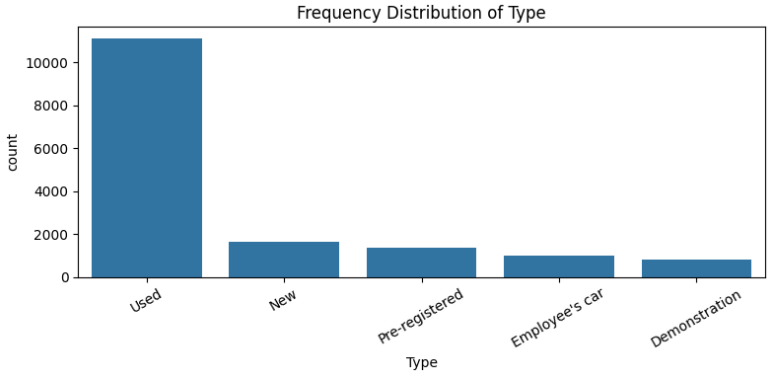
for col in cols:
    plt.figure(figsize=(8, 4))
    sns.countplot(x=car_df[col], order=car_df[col].value_counts().index)
    plt.title(f"Frequency Distribution of {col}")

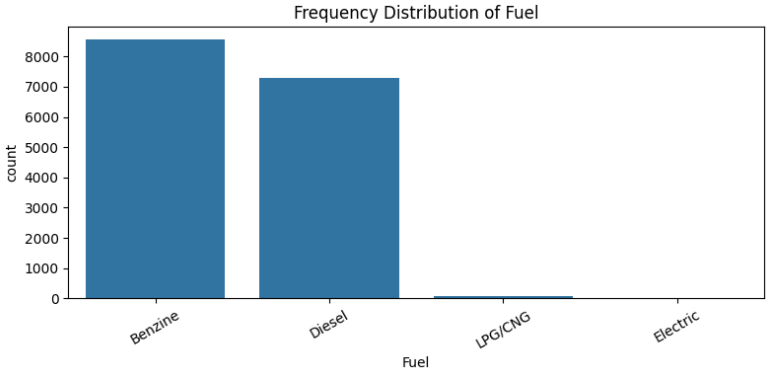
    plt.xticks(rotation=30)
    plt.tight_layout()
    plt.show()
```

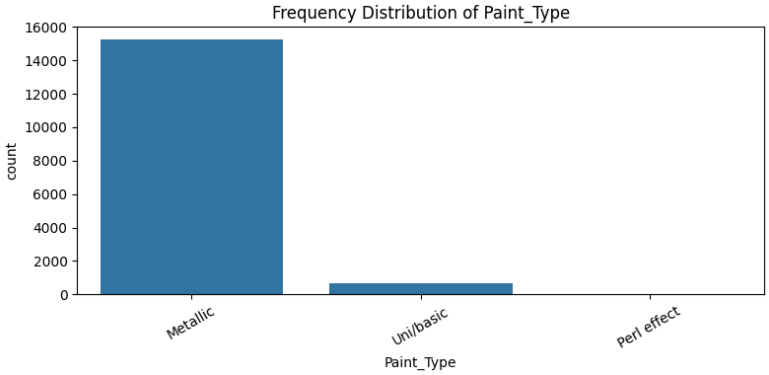

Frequency Distribution of make_model
count
3000
2500
2000
1500
1000
500
0
Audi A3
Audi A1
Opel Insignia
Opel Astra
Opel Corsa
Renault Clio
Renault Espace
Renault Duster
Audi A2
make_model

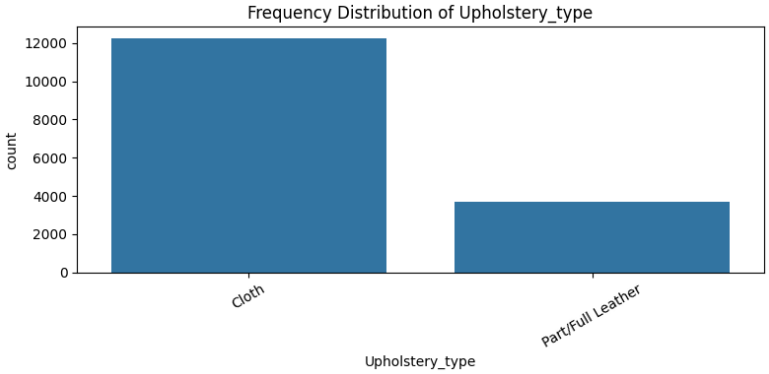

Frequency Distribution of body_type
count
8000
7000
6000
5000
4000
3000
2000
1000
0
Sedans
Station wagon
Compact
Van
Transporter
Off-Road
Coupe
Convertible
body_type

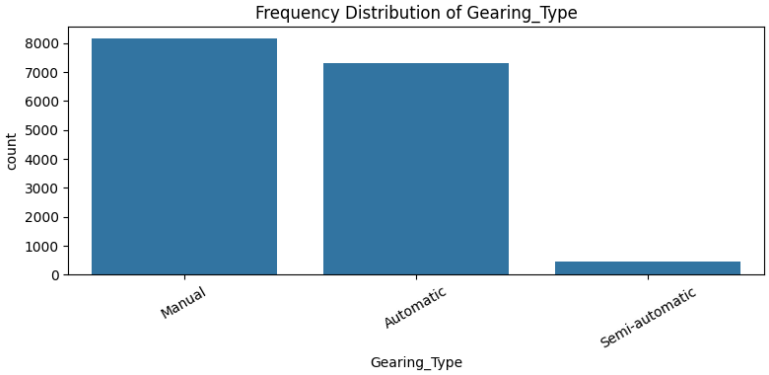

Frequency Distribution of vat
count
14000
12000
10000
8000
6000
4000
2000
0
VAT deductible
Price negotiable
vat

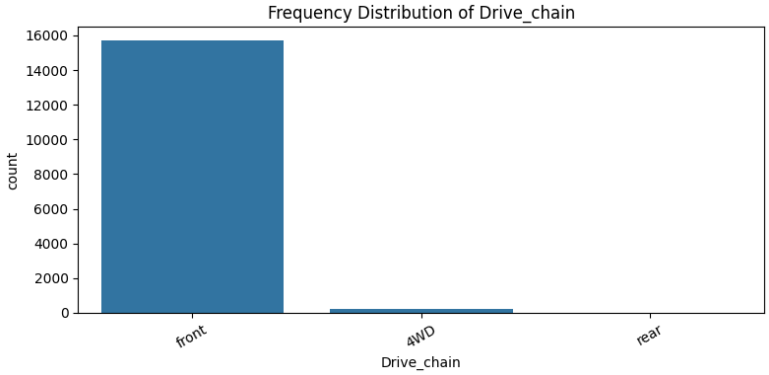

Frequency Distribution of Type
count
10000
8000
6000
4000
2000
0
Used
New
Pre-registered
Employee's car
Demonstration
Type

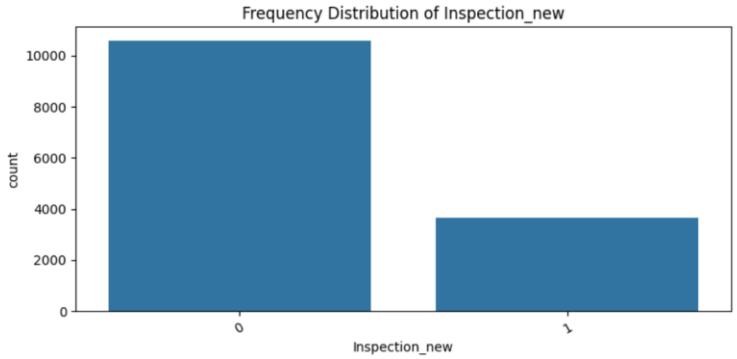

Frequency Distribution of Fuel
count
Benzine
Diesel
LPG/CNG
Electric
Fuel


Frequency Distribution of Paint_Type
count
Metallic
Uni/basic
Perl effect
Paint_Type


Frequency Distribution of Upholstery_type
count
Cloth
Part/Full Leather
Upholstery_type


Frequency Distribution of Gearing_Type
count
Manual
Automatic
Semi-automatic
Gearing_Type

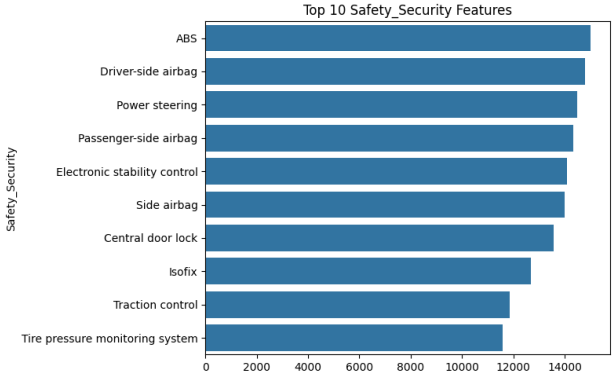


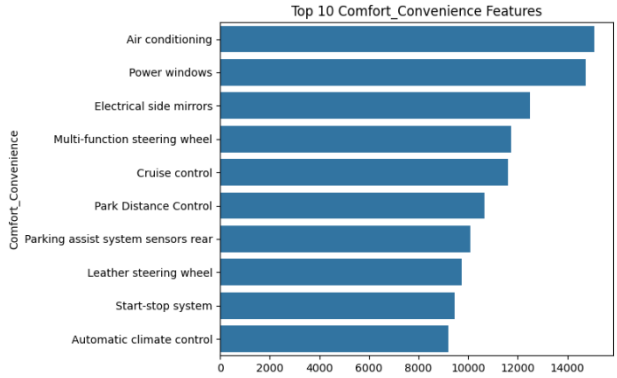


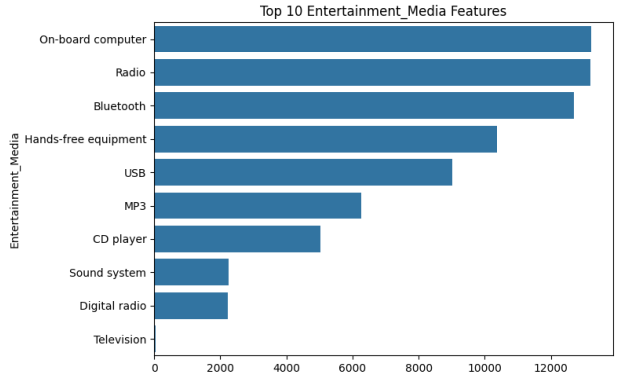

```
multi_cols = ['Safety_Security', 'Comfort_Convenience',
              'Entertainment_Media', 'Extras']

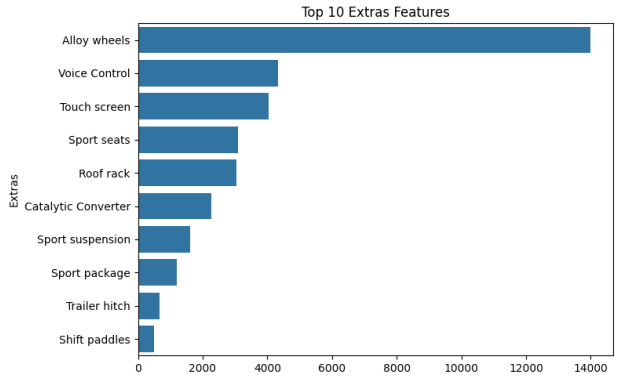
for col in multi_cols:
    expanded = car_df[col].str.split(',').explode().str.strip()
    counts = expanded.value_counts().head(10)

    plt.figure(figsize=(8, 5))
    sns.barplot(x=counts.values, y=counts.index)
    plt.title(f"Top 10 {col} Features")
    plt.tight_layout()
    plt.show()
```









Note: Look carefully at the values stored in columns ["Comfort_Convenience", "Entertainment_Media", "Extras", "Safety_Security"].
Should they be considered categorical? Should they be dropped or handled any other way?

They should be considered and not dropped as they affect pricing.
Keep only top 10 features in the graph to show the frequency distribution

2.1.4 Fix columns with low frequency values and class imbalances

1. **Column Names: Fuel, body Type**

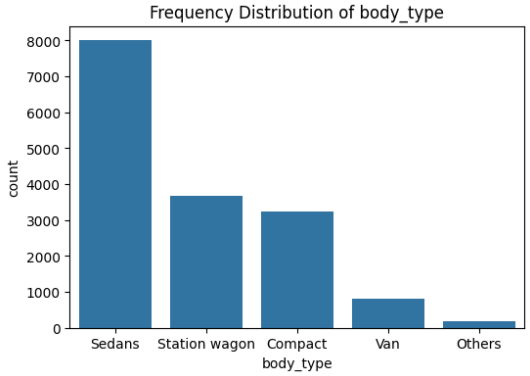
a. Fuel – Categories ‘electric’ and ‘biofuel’ are grouped as others as they constitute less than 1% in Fuel predictor.
This reduces overfitting, dimensionality and keeps major patterns.

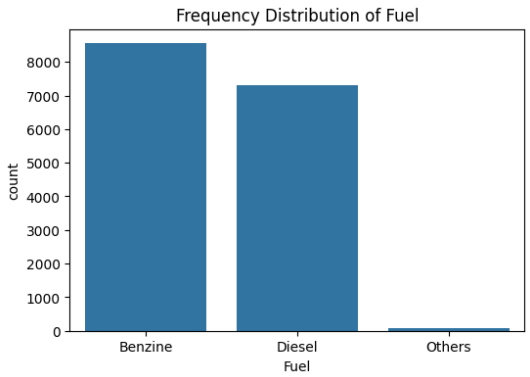
b. Body Type – Categories ‘Transporter’, ‘Off-road’, ‘Coupe’, ‘Convertible’ can be grouped as ‘Others’ as they constitute less than 1% of Body Type Predictor.

```
def group_rare(series, threshold):
    freq = series.value_counts(normalize=True)
    rare_labels = freq[freq < threshold].index
    series.replace(rare_labels, 'Others', inplace=True)
    return series

group_rare(car_df['body_type'], 0.01)
group_rare(car_df['Fuel'], 0.01)

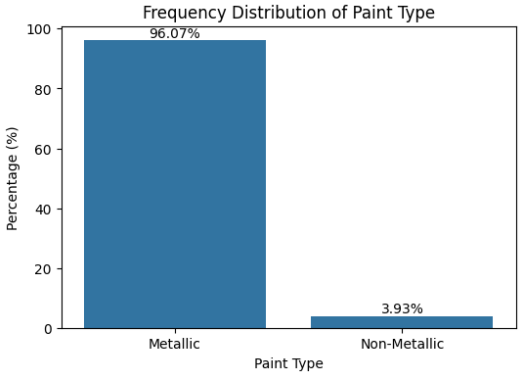
cols = ['body_type', 'Fuel']
for col in cols:
    plt.figure(figsize=(6, 4))
    sns.countplot(x=car_df[col], order=car_df[col].value_counts().index)
    plt.title(f"Frequency Distribution of {col}")
    plt.show()
```



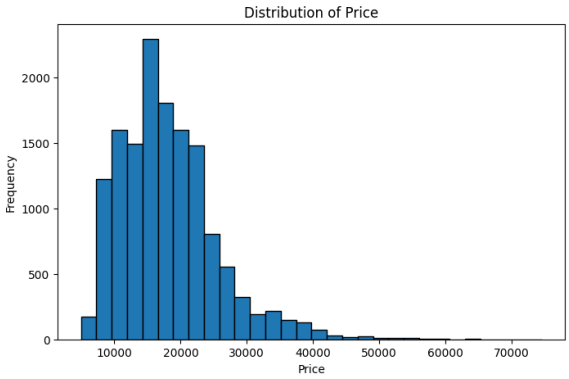


3.**Column Name : Paint**

'Perl Effect' and 'uni/basic' constitute very less percentage of values, hence they are combined and named as non- metallic



2.1.5 Identify target variable and plot the frequency distributions. Apply necessary transformations.



Target feature '**Price'** is Right Skewed.

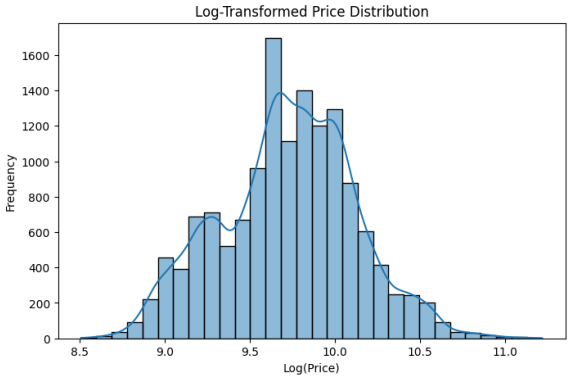
Majority of values are concentrated on the lower end, while a small number of high values extend the tail toward the right.

Log transformation is applied on target feature as it is right skewed.

It compresses larger values more than smaller ones.

It reduces the impact of extreme values, shortens the right tail, and makes the distribution more symmetric and closer to normal.

This improves the performance of regression models by stabilizing variance and reducing the influence of outliers.



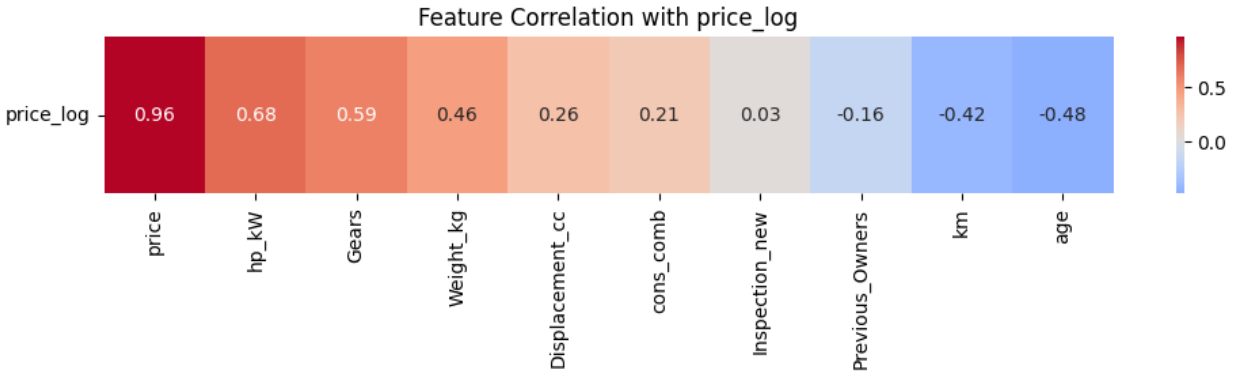
Skew Value before transformation: 1.268475376698342

Skew Values after transformation: -0.03548951812903017

2.2 Correlation Analysis

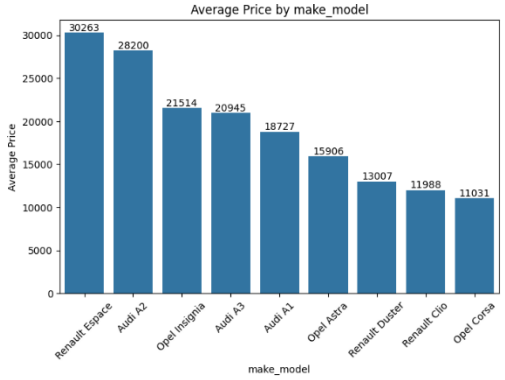
2.2.1 Plot the correlation map between features and target variable.

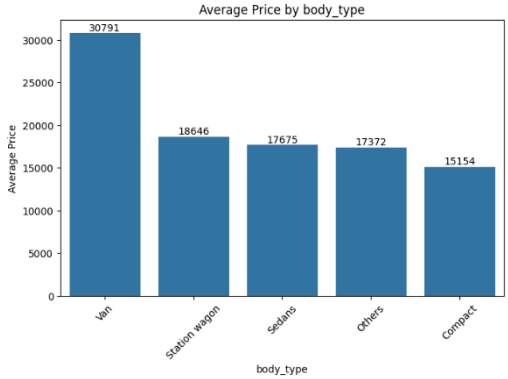
Hp_

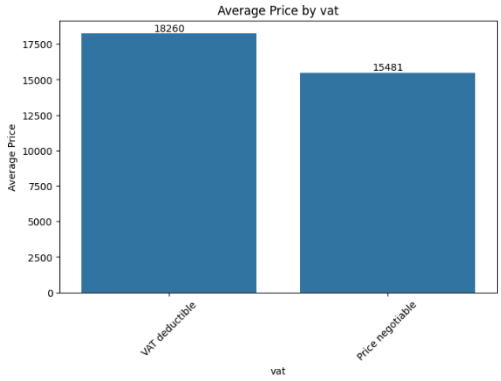


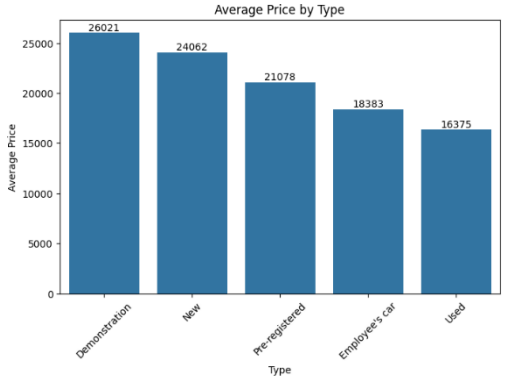
- Engine power (hp_kW), number of gears, and vehicle weight have a strong positive relationship with price
- age and kilometers driven show strong negative correlations,
- inspection status and number of previous owners exhibit weak relationships with price and may have limited predictive value.

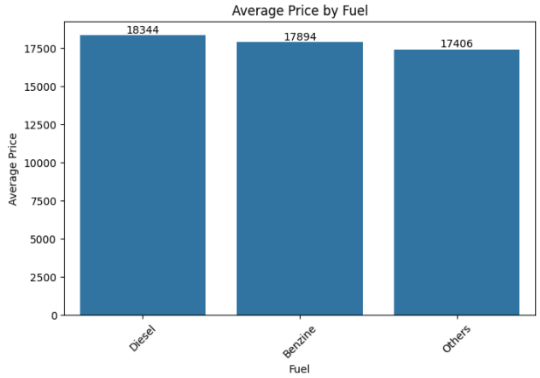
2.2.2 Analyze correlation between categorical features and target variable.

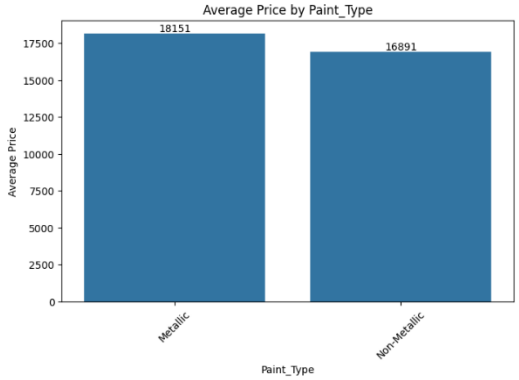

Average Price by make_model
Average Price
make_model
30263
28200
21514
20945
18727
15906
13007
11988
11031
Renault Espace
Audi A2
Opel Insignia
Audi A3
Audi A1
Opel Astra
Renault Duster
Renault Clio
Opel Corsa

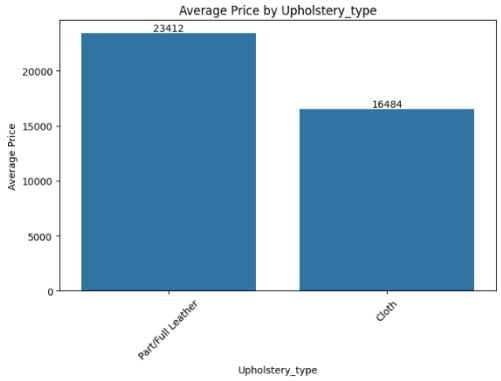

Average Price by body_type
Average Price
body_type
30791
18646
17675
17372
15154
Van
Station wagon
Sedans
Others
Compact

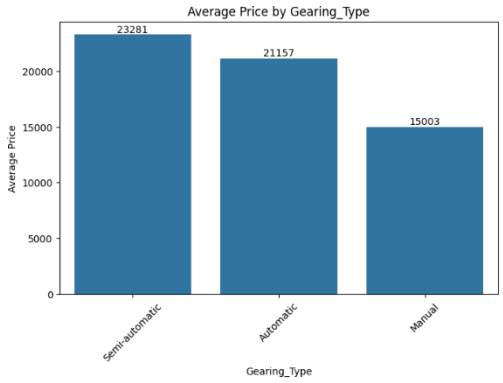

Average Price by vat
Average Price
vat
18260
15481
VAT deductible
Price negotiable

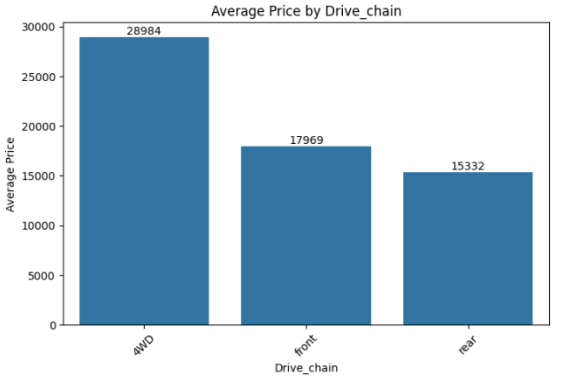

Average Price by Type
Average Price
Type
26021
24062
21078
18383
16375
Demonstration
New
Pre-registered
Employee's car
Used

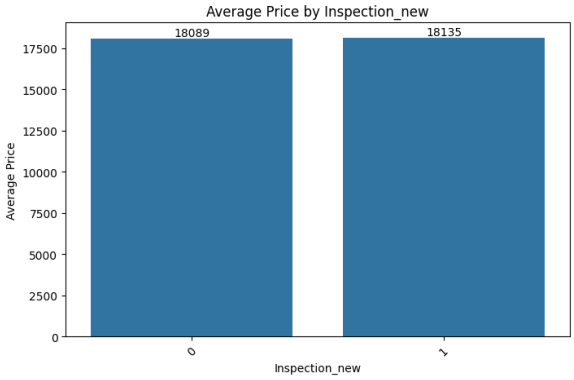

Average Price by Fuel
18344
17894
17406
Average Price
0
2500
5000
7500
10000
12500
15000
17500
Diesel
Benzine
Others
Fuel


Average Price by Paint_Type
18151
16891
Average Price
0
2500
5000
7500
10000
12500
15000
17500
Metallic
Non-Metallic
Paint_Type


Average Price by Upholstery_type
23412
16484
Average Price
0
5000
10000
15000
20000
Part/Full Leather
Cloth
Upholstery_type


Average Price by Gearing_Type
23281
21157
15003
Average Price
0
5000
10000
15000
20000
Semi-automatic
Automatic
Manual
Gearing_Type



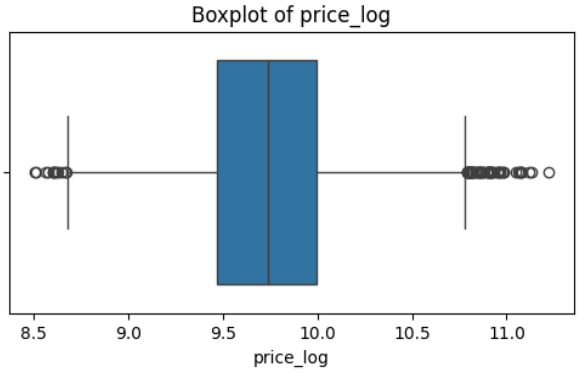


2.3 Outlier Analysis –

2.3.1 Identify potential outliers in the data.

Outliers in the target variable ‘Price_log’

Outliers in the target variable were identified using the IQR method and visualized using boxplots. Since the target variable ‘price’ was log-transformed, the impact of extreme values is reduced, hence not removing any outliers in price_log



Outlier in numerical predictors –

	outlier_count
Previous_Owners	1542
km	657
hp_kW	298
Gears	203
cons_comb	111
Weight_kg	73
Displacement_cc	21
age	0
Inspection_new	0

2.3.2 Handle the outliers suitably.

- Outliers in the target variable were handled using log transformation to reduce skewness and minimize the impact of extreme values.
- Instead of removing outliers in numerical predictors, extreme values are capped to reduce their impact while preserving the dataset size.
- For categorical variables, outlier analysis is not applicable

2.4 Feature Engineering

2.4.1 **Fix any redundant columns and create new ones if needed.**

Multi-label categorical columns['Comfort_Convenience', 'Entertainment_Media', 'Extras', 'Safety_Security'] were replaced with binary encoded features, and the original columns were dropped.

Additionally, new features such as number of feature counts were created to improve the model's predictive capability while reducing dimensionality and redundancy.

Though we get '**previous owners'** as redundant column, has only 1 as value for all the rows. I would like to retain this predictor as it is unique information and does not duplicate any other columns

2.4.2 **Analysis and feature engineering on ['Comfort_Convenience', 'Entertainment_Media', 'Extras', 'Safety_Security']. These columns contain lists of features present. Decide on how to include these features in the predictors.**

Out of these features, we will check the ones which are present in most of the cars or are absent from most of the cars. These kinds of features can be removed as they just increase

the dimensionality without explaining the variance.

Count of original multi label features **'Comfort_Convenience', 'Entertainment_Media', 'Extras', 'Safety_Security'** is created for each feature and Original multi label features are dropped

Highly sparse columns are removed from the data frame

2.4.3 Perform Feature Encoding

Encoding is not required for numerical Columns

Categorical features having single labels must be encoded - Fuel body_type Paint_Type Upholstery_type Gearing_Type Drive_chain vat brand Type_Grouped make_model

Multi label features are handled/encoded using **MultiBinaryLabelizer**

2.4.4 Split the data into training and testing sets.

Data is split is the ratio of 70:30

```
# Split data

#Define X and y

X = car_df.drop(columns=['price', 'price_log'], errors='ignore')
y = car_df['price_log']

#Perform Train-TestSplit
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

#Print the shape of trainand test Dataset

print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)
```

```
X_train shape: (9969, 110)
X_test shape: (4273, 110)
y_train shape: (9969,)
y_test shape: (4273,)
```

Scale the features

Features are scaled by importing StandardScaler.
Both train and test data are scaled

```
# Scale features
from sklearn.preprocessing import StandardScaler

# Initialize scaler
scaler = StandardScaler()

# Fit on training data
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Convert to DataFrame
X_train_scaled = pd.DataFrame(X_train_scaled, columns=X_train.columns, index=X_train.index)
X_test_scaled = pd.DataFrame(X_test_scaled, columns=X_test.columns, index=X_test.index)
```

3. Linear Regression Models

3.1 Baseline Linear Regression model

3.1.1 Build and fit a basic linear regression model. Perform evaluation using suitable metrics.

```
lr_model = LinearRegression()

# Train model
lr_model.fit(X_train_scaled, y_train)


# Predict on test data
y_pred_log_train = lr_model.predict(X_train_scaled)
y_pred_log_test = lr_model.predict(X_test_scaled)

#Convert predictions back to Original Scale
y_pred_train = np.expm1(y_pred_log_train)
y_pred_test = np.expm1(y_pred_log_test)
y_actual = np.expm1(y_test)
```

```
# Evaluate the model's performance

rmse = np.sqrt(mean_squared_error(y_actual, y_pred_test))
r2 = r2_score(y_actual, y_pred_test)

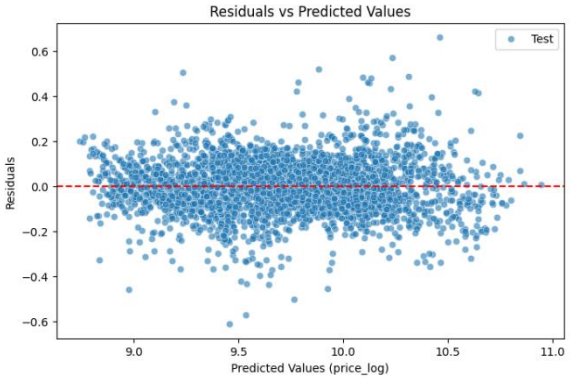
print("RMSE:", rmse)
print("R2 Score:", r2)
```

```
RMSE: 2329.269832942906
R2 Score: 0.9025734828869397
```

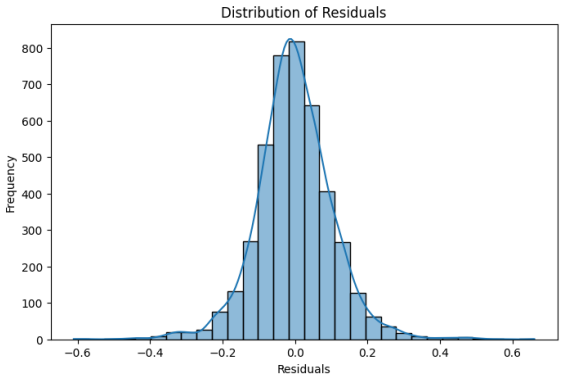
R2 Score is 0.9025

3.1.2 Analyze residuals and check other assumptions of linear regression.

Check for linearity by analyzing residuals vs predicted values



Residuals have constant variance, showing the data is homoscedastic



Errors follow Gaussian or normal distribution indicating linear model is suitable

Check multicollinearity using Variance Inflation Factor (VIF) and handle features with high VIF.

3.2 Ridge Regression Implementation

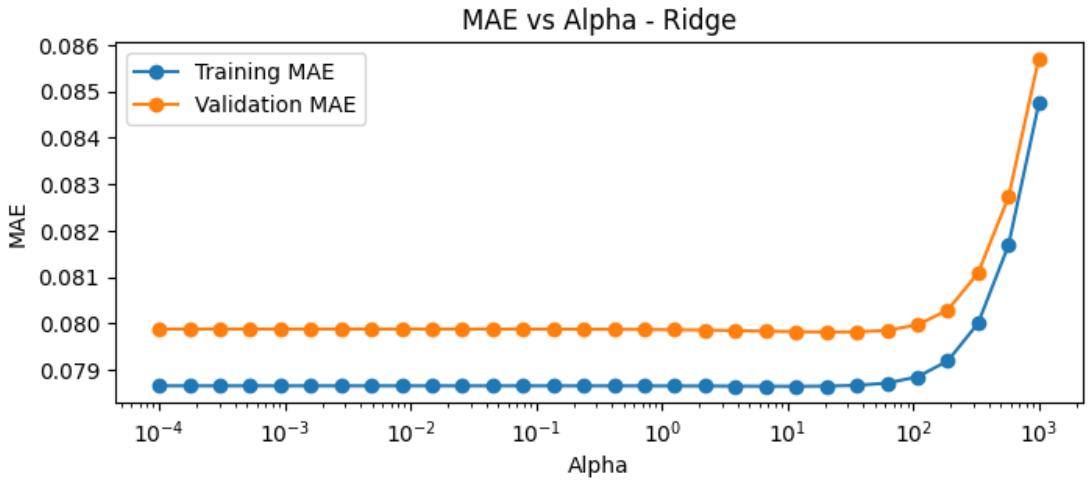
1. Define a list of random alpha values

```
# List of alphas to tune for Ridge regularisation
from sklearn.linear_model import Ridge
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

alpha_list = np.logspace(-4, 3, 30)   # from 0.0001 to 1000
print(alpha_list)
```

```
[1.00000000e-04 1.74332882e-04 3.03919538e-04 5.29831691e-04
 9.23670857e-04 1.61026203e-03 2.80721620e-03 4.89390092e-03
 8.53167852e-03 1.48735211e-02 2.59294380e-02 4.52035366e-02
 7.88046282e-02 1.37382380e-01 2.39502662e-01 4.17531894e-01
 7.27895384e-01 1.26896100e+00 2.21221629e+00 3.85662042e+00
 6.72335754e+00 1.17210230e+01 2.04335972e+01 3.56224789e+01
 6.21016942e+01 1.08263673e+02 1.88739182e+02 3.29034456e+02
 5.73615251e+02 1.00000000e+03]
```

2. Apply Ridge Regularization and find the best value of alpha from the list
Plot train and test scores against alpha



Both training and validation errors remain relatively constant across a wide range of alpha values.
The optimal alpha is selected from the region where validation MAE is low
Extremely small values that provide little regularization and very large values that lead to underfitting are avoided for best alpha value

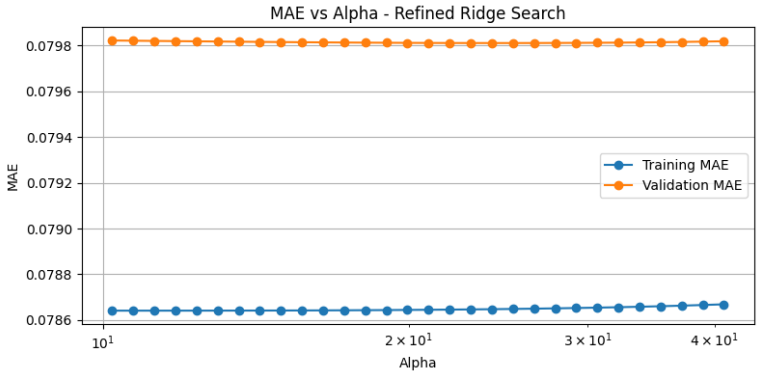
Best ridge alpha value = 20.4336

Absolute Best MAE = 0.0798

Moderate regularization improves model slightly, Model prefers stronger penalty

3. Take a smaller range of alpha, Apply Ridge regression using refined alpha list. Plotting the train and test scores against alpha to find the optimal value of alpha

After obtaining the initial best alpha value of 20.4366, a smaller range was defined around this value to refine the tuning. A range of 10 to 40 was selected to explore values close to the optimal region. This allows a more precise selection of the regularization parameter by focusing on a narrower interval around the previously identified best alpha.



Best refined alpha value: 24.15501437620306

Best score (negative MAE): -0.07981042016857383

Best MAE: 0.07981042016857383

4. Set the best alpha for ridge regression, fit the model and find the coefficients

```
Evaluation on log scale:
MAE (log): 0.07861140059521178
RMSE (log): 0.10650213586350334
R2 (log): 0.9289967712184738

Evaluation on original price scale:
MAE: 1476.9776245100336
RMSE: 2330.7901497679077
R2: 0.9024462606047967
```

3.3 Lasso Regression Implementation –

Lasso is much more sensitive to alpha shrinks coefficients **and sets some to exactly zero**, so we use a smaller range of alpha for Lasso.

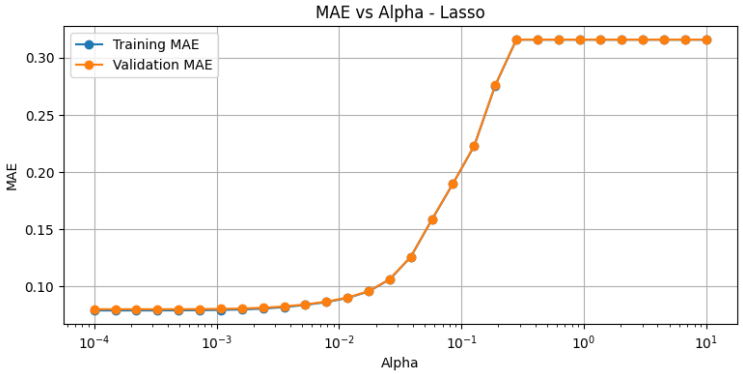
3.3.1 Define a list of random alphas to tune for lasso regularization

```
# List of alphas to tune for Lasso regularisation
import numpy as np

alpha_list = np.logspace(-4, 1, 30)
print(alpha_list)
```

```
[1.00000000e-04 1.48735211e-04 2.21221629e-04 3.29034456e-04
 4.89390092e-04 7.27895384e-04 1.08263673e-03 1.61026203e-03
 2.39502662e-03 3.56224789e-03 5.29831691e-03 7.88046282e-03
 1.17210230e-02 1.74332882e-02 2.59294380e-02 3.85662042e-02
 5.73615251e-02 8.53167852e-02 1.26896100e-01 1.88739182e-01
 2.80721620e-01 4.17531894e-01 6.21016942e-01 9.23670857e-01
 1.37382380e+00 2.04335972e+00 3.03919538e+00 4.52035366e+00
 6.72335754e+00 1.00000000e+01]
```

3.3.2 Apply lasso regularization and find the best value of alpha from the list
Plot train and test scores against alpha



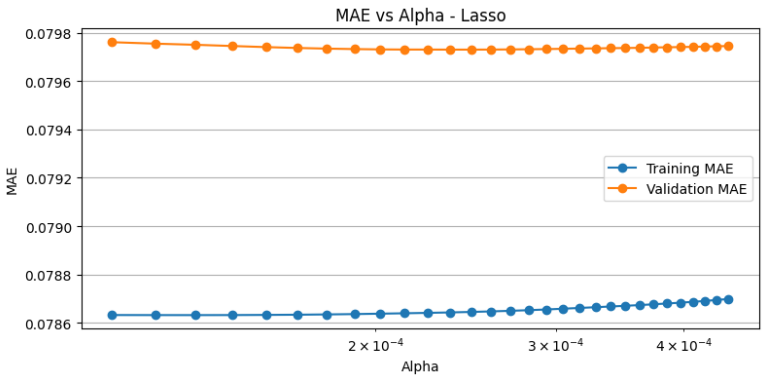
The best region is at very small alpha values and the validation MAE stays at its lowest less than 0.10
very small alpha values give the best validation performance
Model performs well when Lasso regularization is very weak

Best Lasso alpha value: 0.000221221629107045
Best score (negative MAE): -0.07973038348059439
Best MAE: 0.07973038348059439

3.3.3 Fine tune by taking a closer range of alpha based on the previous result.

After obtaining the initial best alpha value for Lasso regression, a smaller range of alpha values was defined around this estimate to refine the tuning process.
The best alpha was selected based on minimum validation MAE.
The final Lasso model was trained using this alpha, and feature importance was assessed through the coefficients, where some coefficients were reduced to zero, indicating automatic feature selection. The model performance was evaluated using MAE, RMSE, and R^2.

Alpha of range 0.00017927 to 0.00044244 is taken for fine tuning



The small difference between training and validation MAE suggests that there is no significant overfitting.
Since validation MAE is almost flat, multiple alpha values in this range provide similar performance, and the final best alpha should be selected using GridSearchCV.

Best alpha value: 0.00024792079124065383
Best score (negative MAE): -0.07973010537575842
Best MAE: 0.07973010537575842

Set the best alpha for Lasso regularization, fit the model, get the co-efficients.
Evaluate Lasso model on test data and find the evaluation metrics

```
Evaluation on log scale:
MAE: 0.07874657384832065
RMSE: 0.1066713177522432
R2: 0.9287710104610185

Evaluation on original scale:
MAE: 1480.04693070379
RMSE: 2336.9550437471894
R2: 0.9019295226930009
```

3.4 **Regularization comparison and Analysis**

3.4.1 Compare the evaluation metrics for each model

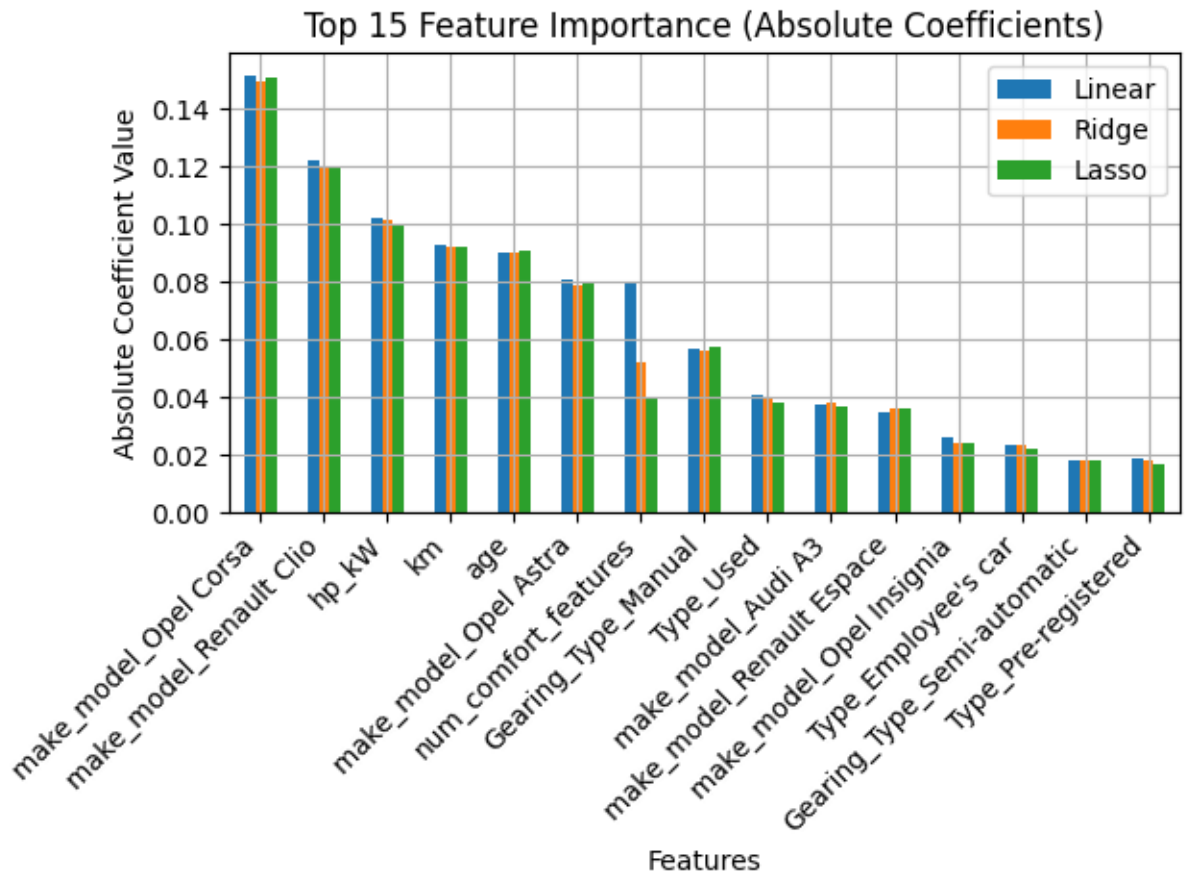
	Model	MAE_log	RMSE_log	r2_price_log	MAE_original	RMSE_original	r2_price_original
0	Linear Regression	0.078566	0.106433	0.929090	1476.332783	2329.269833	0.902573
1	Ridge Regression	0.078611	0.106502	0.928997	1476.977625	2330.790150	0.902446
2	Lasso Regression	0.078747	0.106671	0.928771	1480.046931	2336.955044	0.901930

The comparison of Linear Regression, Ridge Regression, and Lasso Regression shows that all three models perform similarly, with Linear Regression achieving the best results across MAE, RMSE, and R^2.

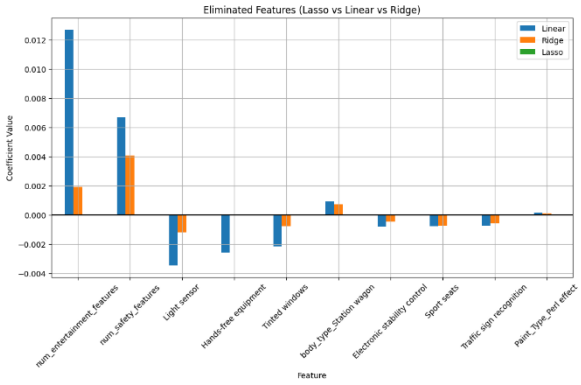
The small differences in performance suggest that the dataset was already well-preprocessed and that overfitting was minimal.

Ridge Regression produced nearly identical results to Linear Regression.
Lasso Regression slightly reduced predictive performance, likely because it removed some useful features while performing feature selection.
Overall, the results indicate that a linear model is sufficient for this problem, and regularization improves interpretability and model stability more than predictive accuracy.

3.4.2 Compare the coefficients for the three models
visualize a few of the largest coefficients.



Visualize the coefficients of features dropped by Lasso



4. Conclusions and Key takeaway -

- Ridge and Lasso **did not significantly improve performance** over Linear Regression
  Metrics across all 3 models are **almost identical**
  Regularization improved model stability and simplicity, NOT predictive accuracy

- There is no significant change as the Data is sufficient and is well-preprocessed, no severe multicollinearity, Linear model is already strong.
  Dataset quality and size are adequate
- **hp_kW (engine power), Gears, Weight_kg** have high positive coefficient value and have more correlation with target variable

  age,km have high negative coefficients and is inversely proportional to target price

  Inspection_new, Previous_owners have no effect on target price

- Irregularities in data were handled
  - by applying log transformation to target Price
  - by capping the outliers of numeric predictors
  - grouping the rare categories in Paint_type, body_type and Fuel
- Feature importance Summary
  - High influence features – hp_kw, Gears, Weight, Age, KM
  - Moderate influence Features – Displacement, Fuel Consumption
  - Low influence Features – Inspection new, Previous_Owners
- Linear Regression performed best, Ridge regression had identical performance and lasso had a slightly lower performance due to removal of some coefficients. Performance metrics MAE, RMSE, R2 Scores were used to determine the performance

- Model gives the price range and not the exact price of the car as exact price depends on Buyer – Seller agreement
- Prediction error varies between 1500-2300 based on MAE representing average prediction error, and RMSE Values which reflects the higher error variations by penalizing larger deviations